

Análise de Algoritmo Aproximativo e Abordagens Exatas para o Problema de Cobertura de Conjuntos

Pedro Lucas Garcia Calais

Departamento de Ciência da Computação
Universidade Federal de Minas Gerais - UFMG
Belo Horizonte, Brasil
pedrolgcalais@ufmg.br

Alice Lessa Ferreira

Departamento de Ciência da Computação
Universidade Federal de Minas Gerais - UFMG
Belo Horizonte, Brasil
alicelessa@ufmg.br

Abstract—O Problema de Cobertura de Conjuntos, ou Set Cover Problem (SCP), é um desafio de otimização combinatória da classe NP-difícil. Este artigo apresenta uma análise comparativa e crítica do desempenho de métodos exatos e aproximados aplicados ao SCP. Implementou-se uma heurística gulosa e abordagens exatas baseadas no paradigma de *Branch-and-Bound*, comparando estratégias de travessia (*Depth-First Search* e *Best-First*) operando em conjunto com dois limitantes inferiores (*Packing Lower Bound* e *Sum-Degree Lower Bound*). A pesquisa foca na análise dos limites teóricos, como a qualidade das soluções da heurística gulosa e na investigação do limiar em que o custo da estratégia exata torna-se proibitivo.

Index Terms—Set Cover, Problema de Cobertura de Conjuntos, Branch-and-Bound, Algoritmos Aproximativos, Limitantes Inferiores, Análise de Desempenho.

I. INTRODUÇÃO

O Problema de Cobertura de Conjuntos (*Set Covering Problem*, ou SCP) é um problema clássico de otimização combinatória. Dado um universo finito $X = \{1, 2, \dots, n\}$ de elementos e uma coleção $\mathcal{S} = \{S_1, S_2, \dots, S_m\}$ de subconjuntos de X , cada um com custo associado $c_j \geq 0$, o objetivo é selecionar uma subcoleção $\mathcal{C} \subseteq \mathcal{S}$ de custo total mínimo tal que todo elemento de X pertença a ao menos um conjunto em $\mathcal{C}$, isto é:

$$\bigcup_{S_j \in \mathcal{C}} S_j = X.$$

O problema é NP-completo e a redução a partir de 3-SAT demonstra que não existe algoritmo de tempo polinomial capaz de resolvê-lo de forma exata, salvo se $P = NP$. Consequentemente, a busca por métodos exatos eficientes ou por aproximações com garantias de qualidade permanece relevante.

Uma abordagem para contornar a intratabilidade computacional é o algoritmo guloso (*greedy*), que, a cada iteração, seleciona o conjunto que cobre o maior número de elementos ainda não cobertos pelo menor custo. Mesmo que não garanta a solução ótima, a solução produzida é maior que a ótima em um fator de no máximo $\ln |X| + 1$ [1].

Para a obtenção de soluções exatas, os algoritmos de *branch and bound* exploram o espaço de soluções de forma sistemática por meio de uma árvore de busca, utilizando limitantes inferiores (*lower bounds*) para podar ramos que

comprovadamente não podem conter a solução ótima. A eficiência do método depende fortemente da qualidade dos limitantes adotados e da estratégia de exploração da árvore, como busca em profundidade (*depth-first search* — DFS) ou busca pelo melhor nó (*best-first search*).

Esta pesquisa investiga experimentalmente o desempenho de diferentes métodos de *branch and bound* para o SCP, comparando estratégias de exploração (DFS e *best-first*) combinadas com dois limitantes inferiores distintos: (*packing lower bound*) e (*sum-degree lower bound*). Os experimentos avaliam o comportamento dos métodos em função de três parâmetros estruturais das instâncias: a densidade dos conjuntos, o número de elementos do universo m e o número de conjuntos n . Adicionalmente, foi analisada a qualidade das soluções produzidas pelo algoritmo guloso em relação ao ótimo obtido pelos métodos exatos, verificando a proximidade com o limite teórico de $\ln |X| + 1$. O objetivo final deste estudo é sustentar a escolha de um método de *branch and bound* adequado para aplicação a instâncias reais de disponibilizadas pela OR-Library [2] em tempo razoável e analisar seu desempenho em tais instâncias.

II. IMPLEMENTAÇÃO DOS MÉTODOS

A implementação original do sistema foi pensada para lidar com instâncias ponderadas do SCP. Contudo, em alinhamento com as especificações da disciplina, o algoritmo foi ajustado para atribuir peso 1 a todos os conjuntos da entrada, tratando a base de dados de forma consistente.

A. Estruturas de Dados

Para garantir um melhor desempenho na exploração da árvore de busca, a classe `SetCoverInstance` modela o universo do problema utilizando máscaras de bits (*bitmasks*). Cada conjunto disponível é traduzido para um único número inteiro onde os bits ativos, ou seja, os bits "1", representam os elementos cobertos pelo conjunto. Essa abordagem permite realizar operações sobre os conjuntos com tempo constante $O(1)$ por meio de operadores bit a bit ($\&$, $|$, $\sim$), como estamos trabalhando com o Problema da Cobertura de Conjuntos sem peso, os custos dos conjuntos foram normalizados para 1, reduzindo o problema a encontrar o número mínimo de subconjuntos necessários para a cobertura.

B. Algoritmo Aproximativo

Para lidar com a resolução do problema em tempo polinomial e fornecer um limitante superior inicial para os métodos exatos subsequentes, foi implementado um algoritmo aproximativo baseado em uma heurística gulosa. O estado global da busca é controlado pela variável *covered_mask*, um inteiro em que cada bit ativo representa um elemento do universo já coberto. A lógica principal está encapsulada no método *calculate_greedy_upper_bound*, o algoritmo opera em um laço de repetição que permanece ativo enquanto a máscara de cobertura for diferente da máscara base (*base_mask*), que representa o universo completo. Para evitar avaliações redundantes, o sistema mantém uma lista secundária (*available_sets*) contendo apenas os índices dos conjuntos que ainda não foram considerados para a solução parcial.

A cada iteração, o algoritmo varre os conjuntos disponíveis em busca daquele que apresente a melhor razão de custo-benefício. Os elementos que um conjunto possui e que ainda não foram cobertos são identificados invertendo a máscara de de elementos já cobertos e aplicando uma interseção lógica com a máscara do conjunto candidato (*sets_masks[idx] & covered_mask*), então, a quantidade de novos elementos é então contabilizada utilizando a função *.bit_count()*. Assim, é possível obter o custo-benefício dividindo o custo dos conjuntos (nesse caso será sempre 1) pela quantidade de novos elementos cobertos e aquele que apresenta a menor razão é selecionado como o melhor candidato da iteração.

Após a escolha, o estado do sistema é atualizado utilizando a operação lógica de disjunção inclusiva para adicionar os elementos do melhor conjunto à máscara de cobertura global (*covered_mask |= sets_masks[best_set_idx]*), o custo do conjunto é adicionado ao total, e o índice do conjunto é removido da lista de variáveis disponíveis. Como mecanismo de segurança, caso existam elementos pendentes, mas não exista nenhum conjunto que cubra os elementos, o algoritmo retorna custo infinito para sinalizar a inviabilidade da instância.

C. Algoritmos Exatos

Para todas as abordagens exatas, a busca é precedida por uma execução do algoritmo aproximativo guloso (*GreedyUB*) e o valor retornado por esta heurística estabelece um limite superior, permitindo que porções substanciais da árvore sejam podadas logo nos primeiros níveis de exploração.

1) *Limitantes Inferiores*: A eficiência da poda no *Branch-and-Bound* depende da capacidade do algoritmo de estimar, a partir de um nó parcial, se ainda existe uma solução viável e qual pode ser seu custo mínimo. Para manter essa verificação separada da estratégia de travessia da árvore, foi criada uma classe base de limitantes que recebe os conjuntos já ordenados pela razão custo-benefício. Nessa classe, um vetor de sufixos cumulativos (*or_suffix*) é calculado de trás para frente, representando os elementos alcançáveis pelos conjuntos restantes a partir de um certo nível da árvore. Assim, ao avaliar um nó, basta combinar os elementos já cobertos com o sufixo correspondente e verificar se é possível atingir todo o universo.

Quando isso não ocorre, o nó é considerado inviável e o ramo é descartado imediatamente, como essa checagem usa apenas operações sobre máscaras de bits previamente calculadas, ela é feita em tempo constante ($O(1)$).

Foram implementadas duas heurísticas de relaxação distintas para prever o custo mínimo necessário, ambas extraídas e fundamentadas no trabalho de Bläsius *et al.* [3]:

- **Packing Lower Bound (*PackingLB*)**: Esta heurística utiliza o conceito de empacotamento de conjuntos (*Set Packing*) para encontrar elementos independentes que não sobreponham soluções. Iterativamente, o algoritmo isola o primeiro elemento não coberto (o bit ativo mais à direita, utilizando a operação *valid_elements & ~valid_elements*) e encontra o custo mínimo necessário para cobri-lo. Em seguida, constrói-se um grafo de conflitos agrupando todos os conjuntos restantes que cobrem este elemento, removendo-os das iterações subsequentes. O somatório dos custos mínimos desses elementos independentes garante um limitante inferior seguro e matematicamente rígido.
- **Sum-Degree Lower Bound (*SumDegreeLB*)**: Esta abordagem realiza uma relaxação mais otimista, assumindo a premissa ideal de que os conjuntos restantes não possuem interseção entre si. O método calcula o "grau útil" de cada conjunto disponível, ou seja, a quantidade exata de novos elementos que ele adiciona à solução parcial por meio da operação (*sets_masks[real_idx] & uncovered_mask*).*.bit_count()*. Os graus úteis são ordenados de forma decrescente e somados cumulativamente até que o total de elementos pendentes seja atingido. O número mínimo de conjuntos (k) necessários para essa soma teórica compõe o valor do limitante inferior.

2) *Estratégias de Travessia*: Para a exploração do espaço de soluções, o implementamos duas abordagens de busca, que diferem, principalmente, no consumo de memória e no critério de expansão dos nós:

- **Busca em Profundidade (DFS)**: A DFS utiliza uma pilha seguindo o princípio LIFO (*Last-In, First-Out*), mantendo apenas os nós do caminho atual e alguns candidatos à expansão, assim, o consumo de memória permanece limitado a $O(n)$. Durante a ramificação, o nó correspondente à exclusão do conjunto é inserido na pilha antes do nó que o inclui, dessa forma, a inclusão é sempre explorada primeiro. Como os conjuntos já estão ordenados pela razão custo-benefício, essa escolha tende a encontrar soluções viáveis mais cedo, permitindo atualizar rapidamente o melhor custo conhecido e aumentar a quantidade de podas realizadas ao longo da busca.
- **Busca Best-First**: O *Best-First* utiliza uma fila de prioridade implementada por meio de um *Min-Heap*, cada nó recebe uma prioridade baseada na soma do custo acumulado com a estimativa fornecida pelo limitante inferior. Assim, a expansão é sempre direcionada para o nó com menor custo estimado. Uma consequência impor-

tante dessa estratégia é que a busca pode ser encerrada assim que o nó mais promissor da fila apresentar um custo estimado maior ou igual ao da melhor solução encontrada, pois nenhum outro nó será capaz de produzir uma solução melhor. No entanto, para manter essa visão global da busca, é necessário armazenar todos os nós da fronteira, o que pode levar a um consumo de memória de até $O(2^n)$ no pior caso, sendo necessário monitorar ativamente o consumo de recursos. Para evitar problemas de ordenação quando dois nós possuem a mesma estimativa, foi utilizado um contador sequencial como critério de desempate na fila de prioridade.

III. GERAÇÃO DE INSTÂNCIAS SINTÉTICAS

Antes de aplicar algum dos algoritmos à base de instâncias reais da OR-Library, foram geradas instâncias sintéticas de escala reduzida com o objetivo de avaliar o desempenho dos métodos em função do tempo de execução e do consumo de memória, variando-se três parâmetros: a quantidade de elementos do universo m , a quantidade de conjuntos n e a densidade d , definida como a fração média de elementos cobertos por conjunto. Neste trabalho, considera-se exclusivamente a variante do SCP em que todos os conjuntos possuem custo unitário, isto é, $c_j = 1$ para todo j .

Além das instâncias aleatórias, foram geradas instâncias adversariais com o objetivo específico de estressar o algoritmo guloso e verificar empiricamente até que ponto sua razão de aproximação se aproxima do limite teórico de $\ln |X| + 1$. A construção dessas instâncias é determinística e inspirada na análise clássica de pior caso do algoritmo guloso para o SCP.

A. Instâncias Aleatórias

As instâncias aleatórias foram geradas por meio de um gerador dedicado, implementado em Python. Para cada instância com parâmetros (m, n, d) , o gerador atribui a cada um dos n conjuntos cada elemento $e \in X$ de forma independente, com probabilidade d . Para garantir a viabilidade da instância, isto é, que todo elemento seja coberto por ao menos um conjunto, qualquer elemento que permaneça descoberto ao final do processo de amostragem é atribuído aleatoriamente a um conjunto escolhido de forma uniforme. A reprodutibilidade dos experimentos é assegurada pelo uso de *seeds* inteiras distintas para o gerador de cada instância.

Foram consideradas as seguintes combinações de parâmetros:

- n fixado em 75, com $m \in \{25, 50, 75, 100, 125\}$;
- m fixado em 75, com $n \in \{25, 50, 100, 125\}$;
- para cada combinação (m, n) , densidades $d \in \{0,1, 0,3, 0,5\}$.

O caso $m = n = 75$ é compartilhado entre os dois grupos, totalizando 27 combinações distintas de parâmetros. Para cada combinação, foram geradas 20 instâncias com *seeds* diferentes, resultando em $27 \times 20 = 540$ instâncias no total.

Cada instância foi salva em um arquivo `.txt` seguindo o formato da OR-Library. O arquivo tem como primeira linha os valores de m e n ; em seguida, os custos dos n conjuntos

dispostos em sequência; e, para cada elemento $e \in X$, o número de conjuntos que o cobrem seguido dos índices (base 1) desses conjuntos.

B. Instâncias Adversariais

Dado o universo $X = \{0, 1, \dots, m-1\}$, a geração parte da divisão de X em dois subconjuntos de índices de paridade distinta:

$$A = \{0, 2, 4, \dots\}, \quad B = \{1, 3, 5, \dots\}.$$

Os dois primeiros conjuntos da instância são exatamente A e B , cada um com custo unitário. Eles constituem a solução ótima de custo 2, pois juntos cobrem todo o universo.

Em seguida, o gerador adiciona iterativamente conjuntos adicionais de forma a prejudicar o critério de seleção do algoritmo guloso. A cada passo, seja $u_A = |A \setminus \text{cobertos}|$ e $u_B = |B \setminus \text{cobertos}|$ as quantidades de elementos ainda não cobertos em A e B , respectivamente. O próximo conjunto adicionado cobre exatamente

$$k = \max(u_A, u_B) + 1$$

dos elementos restantes na ordem crescente. Esse tamanho é calculado de modo que o conjunto recém-adicionado seja sempre ligeiramente superior em cobertura aos conjuntos A e B individualmente, forçando o guloso a preferir esses novos conjuntos e adicionando custo desnecessário. O processo se repete até que todos os elementos estejam cobertos por ao menos um dos conjuntos auxiliares.

O resultado é uma instância em que a solução ótima tem custo 2 (os conjuntos A e B), enquanto o algoritmo guloso, ao privilegiar conjuntos de alta cobertura marginal a cada passo, seleciona um número crescente de conjuntos auxiliares.

Foram geradas instâncias para $m \in \{125, 250, 500, \dots, 4750\}$ (20 valores), totalizando 20 instâncias adversariais. Como a construção não tem aleatoriedade envolvida, cada valor de m produz uma única instância, salvas no formato de arquivo descrito anteriormente.

IV. ANÁLISE EXPERIMENTAL DO ALGORITMO GULOSO

Esta seção analisa a qualidade das soluções produzidas pelo algoritmo guloso em relação ao ótimo obtido pelos métodos de *branch and bound*, avaliando como a razão de aproximação

$$\rho = \frac{\text{custo guloso}}{\text{custo ótimo}}$$

varia em função dos parâmetros densidade, número de elementos do universo m e número de conjuntos n , e como esses resultados se comparam com o limite teórico $\ln |X| + 1$.

A. Instâncias Aleatórias

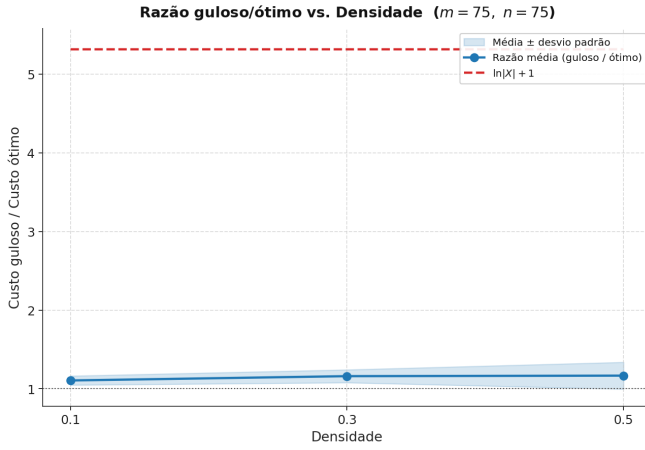


Fig. 1. Razão de aproximação média do algoritmo guloso em função da densidade (com $m = n = 75$). A banda representa a média $\pm$ um desvio padrão e a linha tracejada vermelha indica o limite teórico $\ln |X| + 1$.

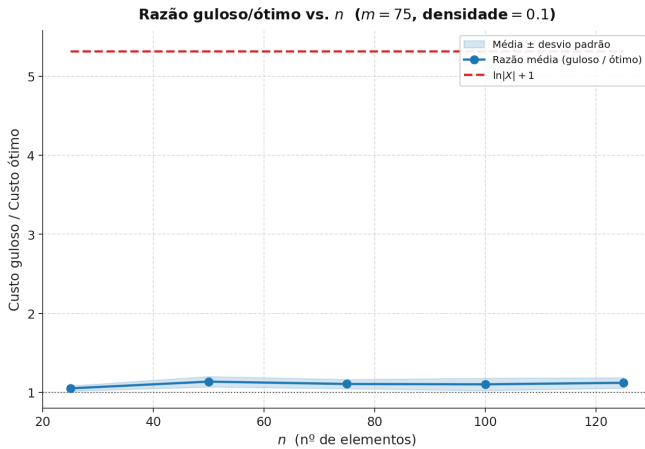


Fig. 2. Razão de aproximação média do algoritmo guloso em função do número de conjuntos (com $m = 75$ e $d = 0, 1$)

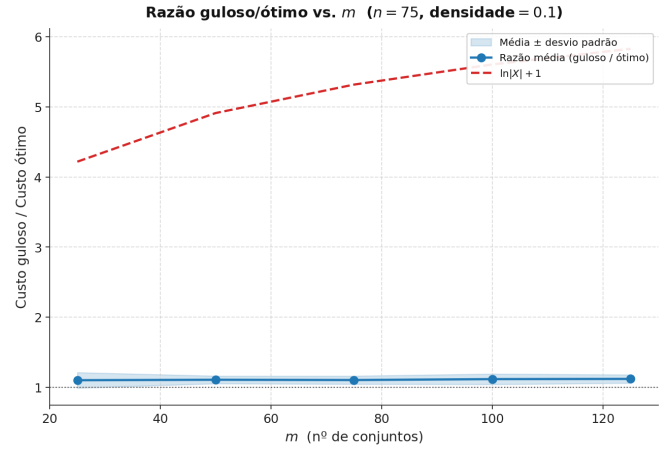


Fig. 3. Razão de aproximação média do algoritmo guloso em função do número de elementos (com $n = 75$ e $d = 0, 1$)

Nas instâncias aleatórias, a razão de aproximação permaneceu consistentemente próxima de 1 em todas as configurações avaliadas, não ultrapassando 1,5 em nenhum caso e mantendo média inferior a 1,3 na grande maioria dos experimentos. Esse comportamento se mostrou estável tanto frente à variação da densidade quanto à variação de m e n , sem tendência de crescimento expressiva em nenhum dos eixos analisados. A diferença em relação ao limite teórico $\ln |X| + 1$, que assume valores entre aproximadamente 3,2 e 5,5 para os parâmetros considerados, é bastante acentuada, evidenciando que o pior caso garantido pela teoria raramente se manifesta em instâncias geradas aleatoriamente.

B. Instâncias Adversariais

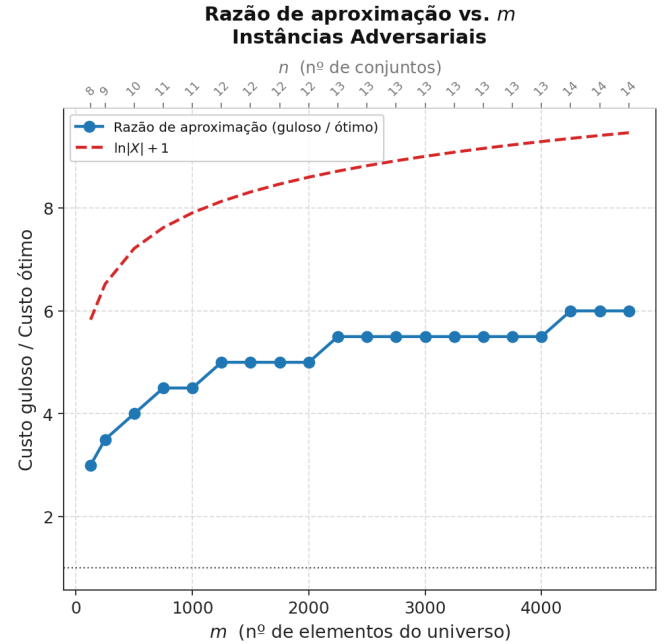


Fig. 4. Razão de aproximação do algoritmo guloso nas instâncias adversariais em função de m . O eixo superior indica o valor de n correspondente a cada instância. A linha tracejada vermelha indica o limite teórico $\ln |X| + 1$.

Nas instâncias adversariais, o comportamento é distinto: a razão de aproximação exibe crescimento logarítmico com m , acompanhando o limite teórico $\ln |X| + 1$ e confirmando que a construção proposta é eficaz em explorar o pior caso do algoritmo guloso. Essa convergência ao limite teórico, contudo, exige condições estruturais muito específicas que dificilmente se verificam em problemas reais.

Um indicador disso é a relação entre m e n nas instâncias adversariais: por construção, n cresce de forma logarítmica em função de m , resultando sempre em n menor que m . Esse comportamento é o oposto do observado nas instâncias reais da OR-Library, onde n é sempre maior que m . Dado que as instâncias adversariais não refletem a estrutura das instâncias reais de interesse, elas não foram incluídas nos experimentos de *branch and bound*, uma vez que não influenciariam a escolha do método a ser aplicado às instâncias da OR-Library.

Os resultados desta seção sugerem, portanto, que o algoritmo guloso é capaz de produzir soluções de boa qualidade, mesmo que sem garantias teóricas fortes, e que o limite $\ln |X| + 1$ constitui um caso muito pessimista.

V. ANÁLISE EXPERIMENTAL DOS MÉTODOS BRANCH AND BOUND

A fim de analisar o desempenho dos métodos de *Branch and Bound* apresentados, cada um deles foi executado sobre todas as instâncias sintéticas aleatórias geradas para este trabalho, utilizando um limite máximo de execução de 30 minutos (1800 segundos) por instância.

O objetivo desta seção é avaliar experimentalmente o comportamento das diferentes combinações de estratégias de travessia e limitantes inferiores implementadas. Para isso, são analisadas não apenas métricas diretamente relacionadas ao desempenho computacional, como tempo de execução e consumo de memória, mas também métricas associadas ao funcionamento interno dos algoritmos, como o número de nós explorados e o número de podas realizadas ao longo da busca.

Essa análise permite comparar a eficiência prática dos métodos propostos, identificar os fatores que mais influenciam seu desempenho e determinar quais combinações se mostram mais adequadas para a resolução exata do SCP nas instâncias reais.

A. Análise de Tempo

Como já discutido, o SCP é um problema NP-completo e, portanto, não são conhecidos algoritmos exatos de tempo polinomial para sua resolução. Nesse contexto, a principal motivação para o uso de técnicas de *Branch and Bound* é reduzir o tempo de execução, tornando viável a obtenção de soluções ótimas.

Para avaliar o impacto das diferentes estratégias implementadas, foram construídos gráficos relacionando o tempo de execução com os parâmetros das instâncias. Para cada combinação de método e conjunto de parâmetros, foi utilizada a mediana dos tempos observados. A escolha da mediana evita distorções causadas pela presença de execuções que atingiram o limite de tempo estabelecido. Além disso, o eixo vertical

foi representado em escala logarítmica, permitindo visualizar resultados com diferença ampla na mesma escala.

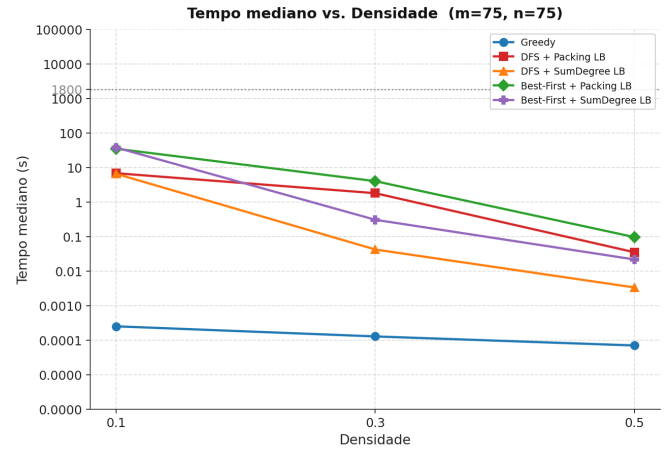


Fig. 5. Tempo mediano de execução em função da densidade dos conjuntos.

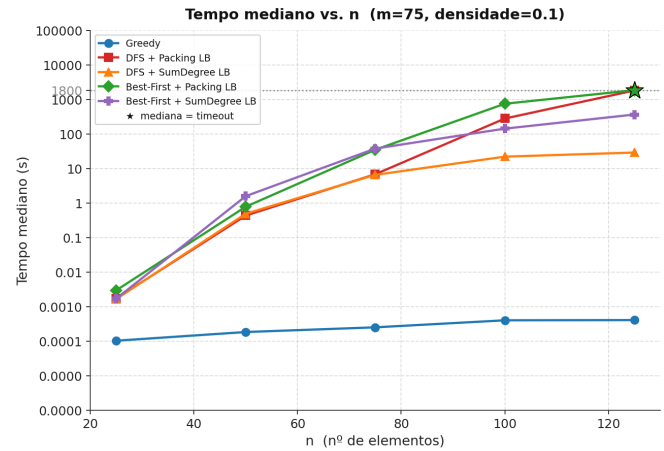


Fig. 6. Tempo mediano de execução em função do número de conjuntos.

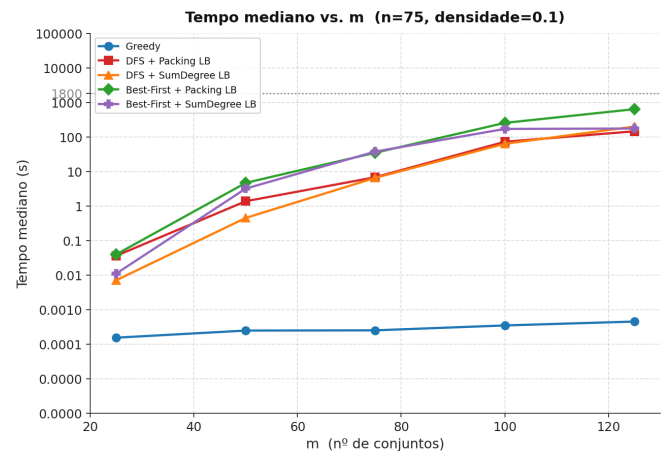


Fig. 7. Tempo mediano de execução em função do número de elementos do universo.

Os resultados mostram uma forte influência da densidade das instâncias no desempenho dos algoritmos. À medida que a densidade aumenta, o tempo de execução diminui significativamente para todos os métodos avaliados. Esse comportamento é esperado, pois conjuntos mais densos tendem a cobrir um número maior de elementos simultaneamente, facilitando a construção de soluções viáveis de baixo custo e permitindo que os limitantes superiores sejam atualizados mais rapidamente. Consequentemente, um número maior de ramos pode ser eliminado precocemente durante a busca.

Em contraste, observou-se crescimento do tempo de execução conforme aumentam os parâmetros m e n . O aumento de m amplia o universo de elementos que devem ser cobertos, enquanto o aumento de n eleva diretamente o tamanho do espaço de busca explorado pelos algoritmos. Como cada conjunto gera uma decisão binária de inclusão ou exclusão, o crescimento de n possui impacto particularmente significativo sobre o tempo de execução. Pode ser observado que o único parâmetro que resultou em *Timeout* como mediana foi $n = 125$.

Quanto à comparação entre as estratégias implementadas, a busca em profundidade (DFS) apresentou desempenho consistentemente superior à busca *Best-First*. Uma possível explicação para esse comportamento é que a DFS tende a encontrar soluções viáveis completas mais rapidamente, permitindo a atualização do limitante superior global mais cedo. Com um limitante superior mais restritivo, um número maior de nós torna-se elegível para poda, reduzindo significativamente o tamanho efetivo da árvore explorada.

Entre os limitantes inferiores avaliados, o *Sum-Degree Lower Bound* obteve desempenho superior ao *Packing Lower Bound*. Como consequência, a combinação *DFS+SumDegreeLB* apresentou consistentemente os menores tempos de execução na maior parte das instâncias analisadas, apresentando-se como a alternativa mais promissora para os experimentos com instâncias reais apresentados na próxima seção.

B. Análise de Consumo de Memória

Além do tempo de execução, a viabilidade de um algoritmo exato depende fortemente de sua eficiência no gerenciamento de memória, para *Branch and Bound*, o consumo de memória está ligado à estratégia de travessia escolhida para explorar o espaço de estados. Para investigar esse aspecto, foi gerado um comparativo da memória máxima consumida (em *Megabytes*) em função do tamanho das instâncias.

Os resultados apresentados na Figura 8 mostram que o consumo de memória foi influenciado principalmente pela estratégia de busca utilizada, enquanto a escolha do limitante inferior teve efeito relativamente pequeno nesse aspecto. A estratégia *Best-First* demandou significativamente mais memória do que a *DFS*, alcançando a ordem de Gigabytes nas maiores instâncias analisadas e, em alguns testes utilizando *Best-First* com *Sum Degree* o algoritmo utilizou toda a memória RAM disponível (12GB) gerando um erro de *OUT_OF_MEMORY* e

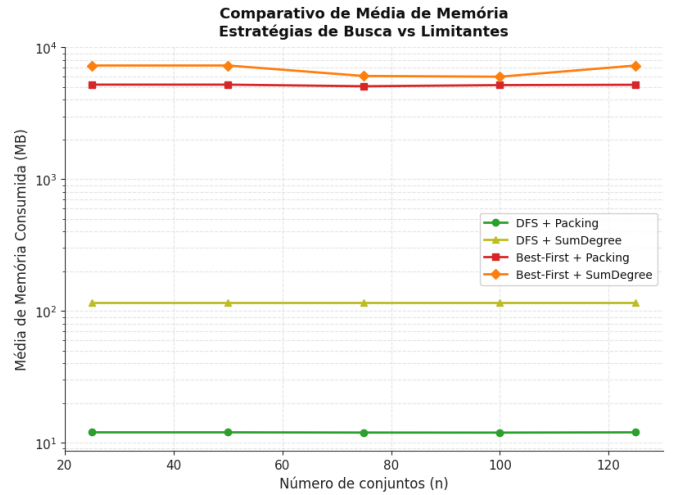


Fig. 8. Consumo máximo de memória (MB) em função do tamanho das instâncias

parando a execução do programa. Essa diferença está associada à própria natureza da busca: enquanto a *DFS* mantém apenas uma pequena parcela da árvore em memória, o *Best-First* precisa armazenar toda a fronteira de busca em uma fila de prioridades para selecionar continuamente o nó com menor custo estimado. Como o número de nós pendentes de exploração pode crescer exponencialmente com o tamanho do problema, o custo de memória da abordagem aumenta rapidamente e passa a representar uma restrição relevante.

A busca em profundidade (DFS) obteve um consumo de memória muito menor, mantendo-se na faixa das dezenas ou centenas de Megabytes, independentemente da dificuldade da instância ou do limitante utilizado. A *DFS* armazena na memória apenas o caminho atual da raiz até o nó corrente (limitado a uma profundidade máxima de n), resultando em uma complexidade espacial linear, o que a torna imune ao gargalo de memória sofrido pelo *Best-First*.

C. Análise da Árvore de Busca e Podas

Para compreender a fundo os resultados de tempo de execução, é fundamental analisar o funcionamento interno dos algoritmos, especificamente o volume do espaço de busca que precisou ser de fato investigado. Para isso, avaliou-se o número absoluto de nós explorados (em escala logarítmica) utilizando a estratégia DFS variando os limitantes *PackingLB* e *SumDegreeLB*.

Embora a análise teórica indique que limitantes mais sofisticados, como o *PackingLB*, deveriam gerar árvores de busca menores, pois sua estimativa mais precisa do custo residual permite podar ramos pouco promissores mais cedo, os resultados experimentais (Figura 9) mostraram que esse comportamento se manifesta apenas nas instâncias de menor complexidade ($n \leq 75$). Nesses cenários, a combinação *DFS+PackingLB* foi capaz de reduzir de forma significativa o número de nós explorados em comparação com as demais configurações.

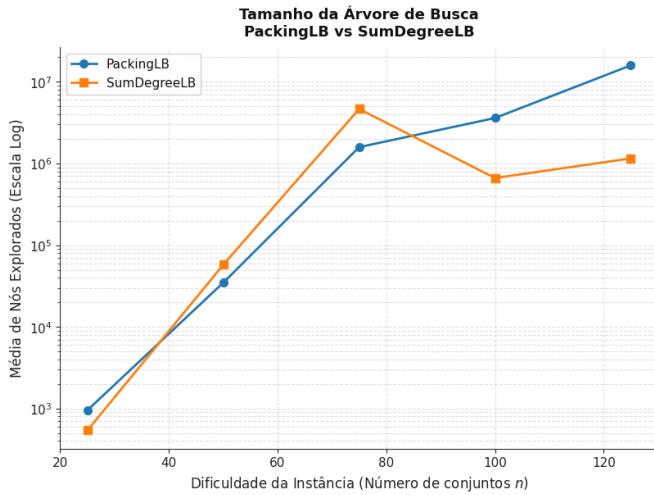


Fig. 9. Proporção de nós explorados por número de conjuntos

No entanto, à medida que o tamanho do problema aumenta ($n \geq 100$), essa vantagem deixou de ser observada: a árvore gerada pelo *PackingLB* cresceu de forma muito mais agressiva, ultrapassando a marca de 10^7 nós, enquanto o *SumDegreeLB* manteve o crescimento da árvore em patamares inferiores.

Esse comportamento evidencia que a eficiência do algoritmo não depende apenas da qualidade teórica do limitante inferior, mas também do custo necessário para calculá-lo e de como esse custo interage com a estratégia de busca. Por exigir apenas operações simples, o *SumDegreeLB* pode ser avaliado rapidamente, permitindo que a *DFS* alcance soluções viáveis em menos tempo e atualize precocemente o melhor custo conhecido. Com um limitante mais restritivo disponível, uma parcela significativa da árvore passa a ser descartada antes mesmo de ser explorada.

Já o *PackingLB* demanda um processamento consideravelmente maior em cada nó. Embora produza estimativas mais precisas, esse custo adicional reduz a velocidade de exploração da árvore e atrasa a descoberta de boas soluções viáveis. Como consequência, durante uma parte significativa da execução o algoritmo opera com um limitante relativamente fraco, permitindo que um número maior de nós seja gerado antes que as podas se tornem muito efetivas.

Os resultados indicam, portanto, que em instâncias mais difíceis do SCP a rapidez proporcionada pelo *SumDegreeLB* acaba sendo mais vantajosa do que a maior precisão do *PackingLB*. Nesse cenário, a redução do tempo gasto em cada nó compensa a menor qualidade da estimativa, resultando não apenas em tempos de execução menores, mas também em árvores de busca significativamente mais compactas.

VI. ANÁLISE DOS RESULTADOS EM INSTÂNCIAS REAIS

Com base nos experimentos preliminares realizados com instâncias sintéticas, a configuração combinando a busca em profundidade com o limitante inferior baseado na soma dos graus (*Depth-First Search* associada ao *Sum-Degree Lower Bound* — *DFS+SumDegreeLB*) foi selecionada para

a avaliação em instâncias reais da *OR-Library*, dado o seu desempenho consistentemente superior. Para investigar os limites computacionais do método exato frente a problemas de maior escala, estabeleceu-se um tempo limite de execução (*timeout*) de 7200 segundos (2 horas) por instância.

Os resultados revelaram a alta complexidade das instâncias reais do Problema de Cobertura de Conjuntos (SCP). Como pode ser observado na Figura 10, 100% dos testes executados por meio da abordagem exata (*Branch-and-Bound*) atingiram o limite de tempo estipulado sem conseguir concluir a varredura completa ou provar a otimalidade da solução dentro do prazo de duas horas.

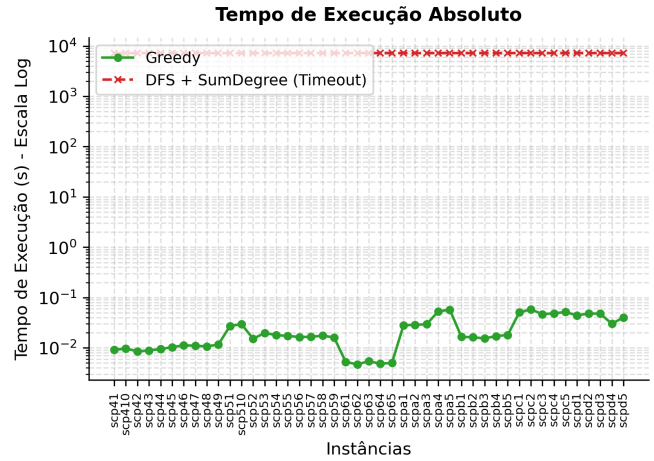


Fig. 10. Comparação do tempo de execução absoluto entre a heurística gulosa e o algoritmo exato (DFS+SumDegreeLB) em escala logarítmica.

O aspecto mais interessante da análise está na qualidade das soluções parciais obtidas antes da interrupção por *timeout*. Em todas as 45 instâncias reais testadas, o melhor custo de cobertura encontrado pelo algoritmo exato no momento do corte foi completamente idêntico ao custo obtido de forma quase instantânea pela heurística gulosa (*Greedy*). Enquanto o procedimento exato despendeu 7.200 segundos abrindo e inspecionando milhões de nós na árvore de busca (conforme ilustrado na Figura 11), o algoritmo aproximativo alcançou exatamente o mesmo resultado em uma fração ínfima de tempo (uma média inferior a 0,03 segundos por arquivo).

A análise do esforço de busca exposta na Figura 11 evidencia que, embora o limitante *SumDegreeLB* tenha sido eficaz em guiar e podar porções da árvore nas instâncias sintéticas, o tamanho do espaço de busca das instâncias reais cresce de forma proibitiva. Mesmo processando dezenas de milhões de nós em subproblemas densos (como nas famílias *scpb6* e *scpa*), a DFS não foi capaz de encontrar ramificações que superassem o teto de qualidade já estabelecido na inicialização pelo método guloso.

Diante do exposto, conclui-se que mesmo a melhor abordagem exata estruturada neste trabalho mostra-se inviável em tempo computacional aceitável para cenários reais dessa escala. Tomando como base a proporção pequena observada entre o resultado da heurística gulosa e o ótimo absoluto obtido

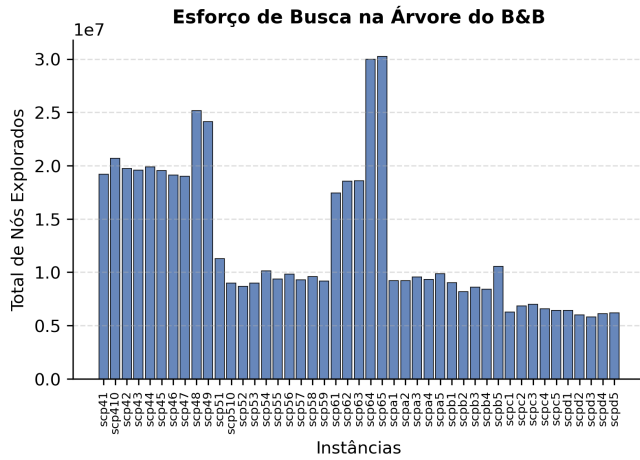


Fig. 11. Esforço computacional do algoritmo de Branch-and-Bound mensurado pelo total de nós explorados antes da interrupção por timeout.

nos testes sintéticos, e assumindo que esse comportamento se mantenha para as instâncias de maior porte da *OR-Library*, o algoritmo *Greedy* pode ser visto como a melhor e mais eficiente opção prática para a resolução do problema analisado.

VII. CONCLUSÃO

Este trabalho apresentou uma análise abrangente e comparativa entre o desempenho de métodos exatos, baseados no paradigma de Branch and Bound, e um algoritmo aproximativo guloso aplicados ao Problema de Cobertura de Conjuntos (SCP). A investigação considerou tanto os limites teóricos quanto o comportamento prático em instâncias sintéticas e reais, evidenciando as nuances estruturais que ditam a eficiência dessas abordagens.

Em relação ao algoritmo aproximativo guloso, os experimentos demonstraram que a heurística é capaz de produzir soluções de alta qualidade em cenários práticos. Nas instâncias sintéticas aleatórias, a razão de aproximação manteve-se consistentemente baixa, não ultrapassando a margem de 1,5 em relação à solução ótima, o limite teórico pessimista de $\ln |X| + 1$ só foi atingido por meio de instâncias adversariais construídas especificamente para forçar o pior caso, as quais exibem propriedades estruturais raras e inversas às encontradas em problemas reais.

A avaliação das abordagens exatas revelou que a viabilidade do Branch and Bound é estritamente dependente das escolhas de arquitetura do algoritmo. Na estratégia de travessia, a Busca em Profundidade (*DFS*) provou-se amplamente superior à estratégia *Best-First*, dado, principalmente, por seu custo exponencial de memória, enquanto a *DFS* opera com um consumo de memória espacial linear que se mantém na faixa de poucos *Megabytes*, a manutenção da fronteira de busca do *Best-First* demandou recursos na ordem de *Gigabytes*, resultando em interrupções por falta de memória (*OUT_OF_MEMORY*) nas instâncias maiores.

Em relação aos limitantes inferiores, foi possível perceber um compromisso claro entre a qualidade teórica da estimativa

e o custo computacional exigido para calculá-la. Embora o *Packing Lower Bound* (*PackingLB*) consiga podar mais nós em instâncias pequenas ($n \leq 75$), seu elevado custo de cálculo retarda a descida da árvore em problemas mais complexos. O *Sum-Degree Lower Bound* (*SumDegreeLB*), por sua vez, demonstrou ser muito mais leve e rápido de ser processado. Essa agilidade permitiu que a *DFS* encontrasse boas soluções viáveis precocemente, estabelecendo um teto de custo rigoroso que resultou em tempos de execução menores e árvores de busca mais compactas nas instâncias mais difíceis ($n \geq 100$).

Em relação aos limitantes inferiores, os resultados evidenciam um compromisso entre a qualidade da estimativa produzida e o custo computacional necessário para obtê-la. Embora o *Packing Lower Bound* (*PackingLB*) tenha se mostrado mais eficaz na redução da árvore de busca em instâncias menores ($n \leq 75$), seu custo de processamento por nó torna-se um fator limitante à medida que a complexidade do problema aumenta. Já o *Sum-Degree Lower Bound* (*SumDegreeLB*), apesar de fornecer estimativas menos refinadas, possui um custo de cálculo significativamente menor, permitindo uma exploração mais rápida da árvore. Esse comportamento se refletiu diretamente no desempenho observado para as instâncias mais difíceis ($n \geq 100$), nas quais o *SumDegreeLB* apresentou tempos de execução menores e árvores de busca mais compactas.

Apesar de a combinação *DFS+SumDegreeLB* ter se consolidado como a arquitetura exata mais eficiente nos testes sintéticos, sua aplicação nas instâncias reais da *OR-Library* demonstrou na prática a intratabilidade do SCP em larga escala. O algoritmo exato atingiu o tempo limite de duas horas em 100% dos cenários avaliados, sem conseguir provar a otimalidade das soluções, em todos os casos testados, o melhor custo parcial encontrado pelo *Branch and Bound* após milhões de nós explorados e horas de processamento foi idêntico ao custo obtido em milissegundos pela heurística gulosa.

Portanto, concluímos que a abordagem exata se torna rapidamente inviável à medida que o tamanho das instâncias aumenta, exigindo volumes de tempo e memória incompatíveis com aplicações reais de maior escala. Em contrapartida, a heurística gulosa apresentou soluções muito próximas do ótimo, mesmo nas instâncias em que o valor ótimo pôde ser determinado, mantendo um custo computacional significativamente menor. Dessa forma, considerando o equilíbrio entre qualidade da solução e eficiência computacional, a abordagem aproximativa mostrou ser a alternativa mais adequada para a resolução de instâncias reais do Problema de Cobertura de Conjuntos.

REFERENCES

- [1] X. Liang, “CSCI3160 Design and Analysis of Algorithms (2025 Fall) — Approximation Algorithms 3: Set Cover and Hitting Set,” Slides de aula, Department of Computer Science and Engineering, The Chinese University of Hong Kong, 2025, baseado em materiais originais do Prof. Yufei Tao.
- [2] J. E. Beasley, “Set covering - OR-Library,” 1990. [Online]. Available: <https://people.brunel.ac.uk/mastjib/jeb/orlib/scpinfo.html>
- [3] T. Bläsius, T. Friedrich, D. Stangl, and C. Weyand, “An efficient branch-and-bound solver for hitting set,” *arXiv preprint arXiv:2110.11697*, 2021.